

Verteilte Systeme, Containerisierung

Vorwort:

Grundlage für diese Aufgabe ist die Präsentation «03_Deployments_Services_ConfigMaps»

Dauer: 80 Minuten

Besprechung: 20 Minuten

Total: 100 Minuten

Block 3 setzt den Kubernetes-Einstieg aus Block 2 voraus. Statt nochmals eine Standard-Applikation aufzubauen, wird nun der erste Dispatch-City-Stand aus bereitgestelltem Code gebaut, deklarativ deployt und als zusammenhängendes System geprüft.

Arbeitsmaterial: öffentliches GitHub-Template «vsc-dispatch-city-03-foundation», verbindlicher Stand «v1.0.0»: <https://github.com/SwitzerChees/vsc-dispatch-city-03-foundation>. Pro Einzelperson oder Zweierteam wird daraus ein eigenes, fortlaufendes Projekt-Repository erstellt.

Setup-Hinweis für Windows und macOS: Verwenden Sie den Cluster «teko-k8s» aus Block 2. Auf Windows ist WSL2 mit Docker-Desktop-Integration am stabilsten; PowerShell funktioniert mit den einzeln aufgeführten Befehlen. Makefile und .sh-Skripte benötigen Git Bash, WSL oder ein installiertes make.

```
k3d cluster list
# vorhandenen, gestoppten Cluster starten:
k3d cluster start teko-k8s
kubectl config use-context k3d-teko-k8s
kubectl get nodes -o wide

# nur falls der Cluster fehlt:
k3d cluster create teko-k8s --agents 2 --wait
```

Falls Port 3000 belegt ist, verwenden Sie z. B. «3002:3000». Die Control API bleibt in Block 3 bei einer Replik, weil jede Replik noch einen eigenen In-Memory-Zustand hätte.

Offizielle Dokumentation

- <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- <https://kubernetes.io/docs/concepts/services-networking/service/>
- <https://kubernetes.io/docs/concepts/configuration/configmap/>
- <https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/>

Aufgabe 1)

Foundation übernehmen und Systemgrenze verstehen

Übernehmen Sie den Starter und prüfen Sie nur die Voraussetzungen aus Block 2. Der Cluster wird wiederverwendet; ein neues NGINX-Lab ist nicht vorgesehen.

- Erstellen Sie aus dem GitHub-Template ein eigenes Repository und verwenden Sie den Release «v1.0.0».
- Prüfen Sie Kontext und Nodes des Clusters «teko-k8s». Erstellen Sie ihn nur neu, falls er nicht mehr vorhanden ist.
- Erklären Sie die Rollen von «dashboard» und «control-api» sowie den Grund für genau eine Control-API-Replik.

Antwort / Notizen:

Repo aus Template, v1.0.0

Cluster war gelöscht, neu erstellt, 3 Nodes Ready, Kontext k3d-teko-k8s

dashboard ist das Frontend auf 3000, control-api das Go Backend mit der Simulation auf 8080

nur 1 Replik weil der Zustand im Speicher ist, bei 2 würden die Stände auseinanderlaufen

Aufgabe 2)

Images bauen und Kustomize-Ausgabe lesen

Bauen Sie aus dem bereitgestellten Code zwei Images, laden Sie diese in den vorhandenen k3d-Cluster und rendern Sie den geplanten Kubernetes-Stand vor dem Deployment.

- Ordnen Sie jedem Image das zugehörige Dockerfile und den Tag «local» zu.
- Importieren Sie beide Images in «teko-k8s» und erklären Sie die Rolle von «imagePullPolicy: IfNotPresent».
- Führen Sie «kubectl kustomize deploy/overlays/block-03-standalone» aus. Welche Ressourcen entstehen durch Base und Overlay?

Antwort / Notizen:

food-delivery-control-api:local aus build/go-service.Dockerfile, food-delivery-dashboard:local aus apps/dashboard/Dockerfile

mit k3d image import geladen, wegen IfNotPresent nimmt Kubernetes das lokale Image und zieht nur wenn es fehlt

Base bringt Namespace, ConfigMap, 2 Services, 2 Deployments, Overlay hängt noch das Label course.teko.ch/block=03 dran

```
# macOS/Linux/Git Bash/WSL:
make images
CLUSTER=teko-k8s make load
kubectl kustomize deploy/overlays/block-03-standalone

# PowerShell oder ohne make:
docker build -t food-delivery-control-api:local --build-arg SERVICE=control-api -f ./build/go-service.Dockerfile .
docker build -t food-delivery-dashboard:local ./apps/dashboard
k3d image import -c teko-k8s food-delivery-control-api:local food-delivery-dashboard:local
kubectl kustomize deploy/overlays/block-03-standalone
```

Aufgabe 3)

Manifeste als Vertrag deployen

Lesen Sie die Referenzen zwischen den Ressourcen und wenden Sie danach den gerenderten Projektstand an.

- Verfolgen Sie für beide Anwendungen: Service-Selector → Pod-Labels sowie «targetPort» → benannter Container-Port.
- Verfolgen Sie bei der Control API: «envFrom» → ConfigMap «simulation-config» und die konfigurierten Probes.
- Deployen Sie das Overlay und belegen Sie mit «rollout status» und «get ... -o wide», dass beide Workloads bereit sind.

Antwort / Notizen:

Selector passt bei beiden zum Pod-Label app.kubernetes.io/name, targetPort http zeigt auf den Port namens http, API 8080, dashboard 3000

envFrom holt APP_MODE und TICK_MS aus simulation-config, Probes /health/ready und /health/live

apply -k, beide rolled out, je 1/1 Running

```
kubectl apply -k deploy/overlays/block-03-standalone
kubectl -n food-delivery get deploy,pods,svc,cm -o wide
kubectl -n food-delivery rollout status deployment/control-api
kubectl -n food-delivery rollout status deployment/dashboard
kubectl -n food-delivery describe deployment/control-api
```

Aufgabe 4)

Service, EndpointSlice und DNS als Kette prüfen

Prüfen Sie nicht nur, ob ein Service existiert, sondern ob er tatsächlich einen bereiten Pod erreicht und im Cluster per DNS auflösbar ist.

- Vergleichen Sie Service-Selector, Pod-Labels und die EndpointSlice des Services «control-api».
- Rufen Sie «/health/ready» aus einem temporären Curl-Pod über Kurzname und vollständigen DNS-Namen auf.
- Erklären Sie, warum der DNS-Name auf dem Host nicht funktioniert und welche Wirkung eine fehlgeschlagene Readiness Probe hätte.

Antwort / Notizen:

Selector und Pod-Label sind beide app.kubernetes.io/name=control-api, EndpointSlice control-api-45xtm hat 10.42.3.4:8080

curl mit control-api und mit control-api.food-delivery.svc.cluster.local, beide status ok

auf dem Mac geht der Name nicht, das DNS gibt es nur im Cluster

Readiness fail: Pod ist in der EndpointSlice nicht mehr ready, kein Traffic, aber kein Neustart

```
kubectl -n food-delivery get svc,endpointslice
kubectl -n food-delivery get pods --show-labels
kubectl -n food-delivery get endpointslice -l kubernetes.io/service-name=control-api

kubectl run dns-test --image=curlimages/curl:8.8.0 -n food-delivery --restart=Never --rm -i -- curl
-fsS http://control-api:8080/health/ready
```

```
# Vollständiger DNS-Name:  
http://control-api.food-delivery.svc.cluster.local:8080/health/ready
```

Aufgabe 5)

ConfigMap, Probes und Rollout nachvollziehen

Ändern Sie das Laufzeitverhalten, ohne ein neues Image zu bauen, und beobachten Sie den Unterschied zwischen ConfigMap-Änderung und Pod-Rollout.

- Erklären Sie «APP_MODE», «TICK_MS», Readiness und Liveness anhand der vorhandenen Manifeste.
- Ändern Sie «TICK_MS» von «500» auf einen gut erkennbaren Wert und wenden Sie die Kustomize-Ausgabe erneut an.
- Prüfen Sie zunächst den bestehenden Pod. Lösen Sie danach bewusst «rollout restart» aus und weisen Sie den neuen Wert in den Startlogs nach.

Antwort / Notizen:

APP_MODE standalone heisst die Simulation läuft in der API, TICK_MS ist ihr Takt in ms

Readiness entscheidet ob der Pod Traffic kriegt, Liveness ob er neu gestartet wird

TICK_MS 500 auf 2000, nach apply läuft der Pod noch mit 500ms

nach rollout restart steht im Log interval:2s

```
kubectl diff -k deploy/overlays/block-03-standalone  
kubectl apply -k deploy/overlays/block-03-standalone  
kubectl -n food-delivery rollout restart deployment/control-api  
kubectl -n food-delivery rollout status deployment/control-api  
kubectl -n food-delivery logs deployment/control-api
```

Aufgabe 6)

Projektstand abnehmen und Architekturgrenze dokumentieren

Nehmen Sie den ersten Dispatch-City-Meilenstein sichtbar und technisch nachvollziehbar ab.

- Öffnen Sie Dashboard und Control API per Port-Forward, starten Sie die Simulation und beschreiben Sie den sichtbaren Ablauf.
- Dokumentieren Sie je eine Evidenz für Image/Deployment, Service/DNS sowie ConfigMap/Probes/Rollout.
- Begründen Sie, weshalb die In-Memory-Control-API bei einer Replik bleibt und welche späteren Komponenten diese Grenze auflösen.

Antwort / Notizen:

Port-Forward auf 3000 und 8081, Simulation gestartet, tickt im TICK_MS Takt, Events kommen live per SSE ins Dashboard

deploy 1/1 mit :local Image, curl über den DNS Namen ok, nach Restart TICK_MS 2s im Log

jede Replik hätte ihren eigenen Zustand im Speicher, darum nur 1. Später lösen das RabbitMQ und Postgres mit CloudNativePG

```
kubectl -n food-delivery get deploy,pods,svc,endpointslice,cm

# Terminal 1:
kubectl -n food-delivery port-forward service/dashboard 3000:3000

# Terminal 2:
kubectl -n food-delivery port-forward service/control-api 8081:8080

# Browser:
http://127.0.0.1:3000
http://127.0.0.1:8081/health/ready
```

Aufräumen

Löschen Sie nur den Namespace «food-delivery». Der Cluster «teko-k8s» wird für Block 4 weiterverwendet und nur bei einem vollständigen Reset entfernt.

```
kubectl delete namespace food-delivery
# Der Cluster bleibt für Block 4 bestehen.
# Nur bei vollständigem Reset:
k3d cluster delete teko-k8s
```